TP UNITY

Système de compétences modulaire pour un RPG

Durée indicative : 4 à 5 heures

Objectif général
Développer un système de compétences modulaire à partir d'un projet Unity existant : données configurables, projectiles, dégâts, événements et interface utilisateur.

Le projet de départ est volontairement partiellement préparé et imparfaitement organisé. Votre première tâche consiste à le comprendre, à le remettre en ordre et à mettre en place un suivi Git exploitable pour la correction.

1. Prise en main et organisation du projet

1.1 Analyser le projet existant

Explorez le projet Unity et identifiez les personnages, composants, Animator Controllers, animations disponibles, Prefabs, éléments de scène, scripts et ressources graphiques. Comprenez le rôle des principaux éléments avant de commencer à développer.

Ne supprimez pas un élément simplement parce que vous ne savez pas à quoi il sert : vérifiez ses références et son utilisation.

1.2 Créer et utiliser un dépôt Git

Créez un dépôt Git pour le projet. Il servira pendant toute la durée du TP et constituera le support de votre rendu. Ajoutez un fichier .gitignore adapté à Unity afin d'exclure les dossiers générés automatiquement (par exemple Library, Temp ou Obj).

Règle obligatoire pour la correction : vous devez effectuer au minimum un commit à chaque étape validée. Chaque commit doit avoir un message explicite, clair et identifiable, qui permet de comprendre immédiatement ce qui a été terminé. Exemple : « README prêt pour correction ». Les messages vagues comme « update », « fix » ou « travail » ne sont pas acceptés. Ne regroupez pas tout votre travail dans un seul commit final.

Les commits doivent matérialiser les étapes importantes : organisation du projet, README, données des compétences, système de lancement, gestion de vie, interface, etc. Le nombre de commits peut donc être supérieur au nombre d'étapes du sujet.

1.3 Nettoyer et organiser le projet

- Regroupez les scripts dans des dossiers appropriés.
- Séparez les scènes, Prefabs, ressources graphiques, animations, matériaux et effets.
- Adoptez une convention de nommage cohérente.
- Vérifiez les doublons et les fichiers réellement inutilisés avant toute suppression.
- Déplacez les fichiers avec précaution pour ne pas casser les références Unity.

1.4 Compléter le README.md

Le README doit permettre à une autre personne de comprendre rapidement le projet. Il devra contenir :

- Présentation : fonctionnement général du système développé.
- Organisation : principaux dossiers et leur contenu.
- Fonctionnalités : liste et explication des fonctionnalités implémentées.

- Installation : version de Unity et dépendances nécessaires.
- Assets externes : nom, provenance, lien vers la page d'origine et explication de l'utilisation de chaque ressource.

Le README doit être mis à jour au fil du travail, et pas seulement décrire le projet dans son état initial.

2. Comprendre le système attendu

À la fin du TP, le joueur doit pouvoir déclencher plusieurs compétences depuis une barre d'actions. Lorsqu'une compétence est utilisée :

- Le personnage déclenche son animation d'attaque.
- Un projectile est créé et se déplace vers la cible.
- À l'arrivée du projectile, la cible subit des dégâts.
- Un effet visuel apparaît à l'impact.
- La barre de vie de la cible est mise à jour.
- La compétence entre en recharge et son bouton indique son indisponibilité.

3. Créer les données des compétences

Concept : ScriptableObject. Créez une classe AbilityData héritant de ScriptableObject. Elle représente la définition d'une compétence et contient les données nécessaires à son fonctionnement et à son affichage.

Donnée	Description
Name	Nom de la compétence
Description	Description affichée ou consultable
Icon	Icône de la compétence
Damage	Quantité de dégâts infligés
Cooldown	Temps de recharge
Projectile Speed	Vitesse du projectile
Effect Color	Couleur associée à la compétence
Projectile Prefab	Prefab du projectile à instancier
Impact Prefab	Prefab de l'effet à l'impact
Animation Trigger	Nom du trigger d'animation à déclencher

Créez au minimum trois ressources AbilityData différentes. Important : le ScriptableObject décrit une compétence, mais ne doit pas stocker son état temporaire (par exemple le temps de recharge restant).

4. Développer le système de compétences

Concepts : MonoBehaviour, Prefabs, instanciation et gestion d'état. Créez un composant AbilityManager chargé de gérer les compétences du personnage.

- Contenir une liste de ressources AbilityData.
- Initialiser les compétences disponibles.
- Permettre de déclencher une compétence à partir de son identifiant ou de son instance.
- Vérifier qu'une compétence est disponible avant son exécution.
- Gérer les temps de recharge indépendamment des ScriptableObjects.

Distinguez donc la définition d'une compétence (ScriptableObject) de son état d'exécution, propre au personnage et à la partie en cours.

Déclenchement : vérifiez la disponibilité, déclenchez l'animation, instanciez le projectile, déplacez-le vers la cible, appliquez les dégâts à son arrivée, jouez l'effet d'impact et lancez le cooldown. Le déplacement doit être réalisé avec DOTween, pas avec une boucle Update(). Utilisez notamment OnComplete() pour gérer l'arrivée du projectile.

5. Gérer les points de vie et les événements

Concepts : System.Action, événements. Créez un composant Health contenant les points de vie maximum et actuels, ainsi que des méthodes pour infliger et restaurer des points de vie. Gérez la mort lorsque les points de vie atteignent zéro.

Le composant Health ne doit pas connaître l'interface qui affiche les points de vie. Utilisez des événements System.Action pour signaler :

- Les changements de points de vie, en transmettant les valeurs actuelles et maximales.
- La mort du personnage.

Les autres composants doivent pouvoir s'abonner à ces événements pour réagir aux changements, sans créer de dépendance directe avec Health.

6. Créer l'interface utilisateur

6.1 Barre de vie

La barre de vie de la cible n'est pas fournie : vous devez la concevoir et l'intégrer vous-mêmes. Elle devra comporter au minimum une barre de fond, une barre représentant les points de vie actuels et, si possible, une indication numérique.

Abonnez-vous à l'événement du composant Health pour mettre à jour l'interface. Utilisez DOTween pour animer la diminution ou la restauration de la barre plutôt que de modifier brutalement sa valeur.

6.2 Barre de compétences

Créez une barre d'actions permettant d'utiliser les compétences du joueur. Les boutons doivent être générés dynamiquement à partir de la liste de compétences de l'AbilityManager : ne créez pas manuellement un bouton par compétence dans la scène.

- Afficher l'icône de chaque compétence.
- Demander au système de compétences de déclencher la compétence correspondante lors du clic.
- Afficher une indication visuelle du temps de recharge.
- Désactiver le bouton pendant le cooldown et le rendre à nouveau disponible à la fin.

L'ajout d'une nouvelle ressource AbilityData doit suffire à faire apparaître automatiquement un nouveau bouton.

7. Utiliser les lambdas

Concepts : delegates, System.Action et expressions lambda. Utilisez des lambdas notamment pour :

- Associer une action à un bouton généré dynamiquement.
- S'abonner à un événement en utilisant une lambda.
- Transmettre des paramètres à une méthode sans créer une méthode intermédiaire pour chaque cas.

Attention à la durée de vie des abonnements : désabonnez-vous des événements lorsque cela est nécessaire, notamment lors de la destruction d'un objet.

8. Fonctionnalités bonus

- Réaction à l'impact : déclencher une animation Hit lorsque la cible reçoit des dégâts, si elle est disponible.
- Gestion de la mort : déclencher l'animation de mort, empêcher les nouvelles attaques et désactiver ou adapter la barre de vie.
- Feedback visuel : ajouter un flash, un changement de couleur ou une animation de bouton.
- Plusieurs cibles : sélectionner différentes cibles et diriger les projectiles vers la cible sélectionnée.
- Architecture évolutive : proposer une architecture pouvant accueillir soin, buff, attaque de zone, etc.

9. Contraintes techniques

- Les données des compétences sont définies avec des ScriptableObjects.
- Les cooldowns sont gérés indépendamment des ScriptableObjects.
- Les projectiles et impacts sont instanciés depuis des Prefabs.

- Le déplacement des projectiles utilise DOTween.
- Les changements de points de vie sont communiqués par des événements System.Action.
- Les boutons de compétences sont générés dynamiquement.
- Les interactions font appel à des expressions lambda.
- Les composants restent aussi indépendants que possible.

Par exemple, Health ne doit pas modifier directement la barre de vie de l'interface.

10. Ressources fournies

Personnages et animations

KayKit Adventurers — Personnages 3D low-poly utilisés dans le projet.
https://kaylousberg.itch.io/kaykit-adventurers

KayKit Character Animations — Animations des personnages, notamment déplacement et combat.
https://kaylousberg.itch.io/kaykit-character-animations

Les personnages sont déjà placés dans la scène. L'un d'eux sert de mannequin et représente la cible. Une partie des animations et de leur configuration est préparée, mais toutes les animations ou tous les états nécessaires ne sont pas forcément disponibles : vérifiez l'existant avant de les utiliser.

Effets visuels

Magic Effects FREE — Pack d'effets magiques à explorer pour trouver des effets adaptés aux compétences.
Page du pack

Hit Effects FREE — Pack d'effets d'impact à utiliser lors de la réception des projectiles.
Page du pack

Ces packs fournissent des effets visuels existants. Identifiez les Prefabs pertinents et intégrez-les correctement. Ne supposez pas qu'un pack contient nécessairement un projectile adapté : vérifiez son contenu.

Outil

DOTween — Bibliothèque d'animations pour les déplacements et l'interface.
https://dotween.demigiant.com/

11. Organisation indicative du travail

Étape	Travail	Durée
1	Analyse, Git, nettoyage et README	45 min
2	Création des données de compétences	30 min
3	Développement de l'AbilityManager	45 min
4	Projectiles et application des dégâts	45 min
5	Health et événements	30 min
6	Interface : barres de vie et de compétences	60 min
7	Tests, corrections et documentation	30 min

La durée est indicative. Obtenez d'abord un système fonctionnel avant de vous lancer dans les bonus.

12. Livrables

- Le dépôt Git du projet, contenant les fichiers nécessaires à son ouverture dans Unity.
- Un historique Git avec au minimum un commit par étape validée, et des messages explicites permettant la correction.
- Un README.md complet : présentation, fonctionnalités, organisation, installation et assets externes.
- Un projet Unity fonctionnel permettant de tester les compétences et leurs effets.

Le projet doit pouvoir être ouvert et exécuté dans la version de Unity indiquée dans le README, sans manipulation particulière autre que l'ouverture du projet.

13. Barème d'évaluation

Critère	Points
Analyse du projet, organisation et nettoyage	2
Utilisation de Git et qualité de l'historique	1
README et documentation des ressources externes	2
Architecture des données avec ScriptableObjects	2
AbilityManager et gestion des cooldowns	3
Projectiles et déplacement avec DOTween	2
Points de vie et événements System.Action	2
Interface et génération dynamique des boutons	3
Utilisation des lambdas et qualité du code	2
Fonctionnalités bonus	1
TOTAL	20

Un système qui fonctionne mais dont l'architecture est mal conçue, fortement couplée ou difficile à faire évoluer ne pourra pas obtenir tous les points. La qualité du code, la clarté de l'organisation et la capacité à expliquer vos choix techniques font partie intégrante de l'évaluation.